

Fashion MNIST Classification & Clustering

Complete Project Documentation

Table of Contents

1. [Project Overview](#project-overview)
2. [System Requirements](#system-requirements)
3. [Installation Guide](#installation-guide)
4. [Architecture Overview](#architecture-overview)
5. [Detailed Module Documentation](#detailed-module-documentation)
6. [Usage Guide](#usage-guide)
7. [Results & Performance](#results--performance)
8. [Troubleshooting](#troubleshooting)
9. [References](#references)

Project Overview

Purpose

This project implements a complete machine learning pipeline for classifying clothing items from the Fashion MNIST dataset using both supervised and unsupervised learning approaches.

Key Features

- **Feature Extraction**: Uses pre-trained ResNet18 for transfer learning
- **Supervised Classification**: Logistic Regression with hyperparameter tuning
- **Unsupervised Clustering**: K-Means clustering analysis
- **Comprehensive Evaluation**: Multiple metrics and visualizations
- **Production-Ready**: Includes caching, progress tracking, and error handling

Dataset

Fashion MNIST contains 70,000 grayscale images (28×28 pixels) across 10 clothing categories:

- T-shirt/top (0)
- Trouser (1)
- Pullover (2)
- Dress (3)
- Coat (4)
- Sandal (5)
- Shirt (6)
- Sneaker (7)
- Bag (8)
- Ankle boot (9)

Split:

- Training: 50,000 images

- Validation: 10,000 images
- Test: 10,000 images

System Requirements

Hardware

Minimum:

- CPU: 4 cores
- RAM: 8 GB
- Storage: 5 GB free space

Recommended:

- CPU: 8+ cores
- RAM: 16 GB
- GPU: NVIDIA GPU with 4GB+ VRAM (CUDA support)
- Storage: 10 GB free space

Software

- Python 3.8 or higher
- CUDA Toolkit 11.0+ (for GPU acceleration)
- pip package manager

Installation Guide

Step 1: Clone/Download Project

```
``bash
# Create project directory
mkdir fashion_mnist_project
cd fashion_mnist_project
``
```

Step 2: Install Dependencies

```
``bash
pip install torch torchvision scikit-learn matplotlib seaborn pandas tqdm numpy scipy
``
```

Package Versions (Recommended):

- torch >= 1.12.0
- torchvision >= 0.13.0
- scikit-learn >= 1.0.0
- matplotlib >= 3.5.0
- seaborn >= 0.11.0
- pandas >= 1.4.0
- numpy >= 1.21.0

- scipy >= 1.7.0
- tqdm >= 4.62.0

Step 3: Verify Installation

```
```python
import torch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```
```

Step 4: Create Required Directories

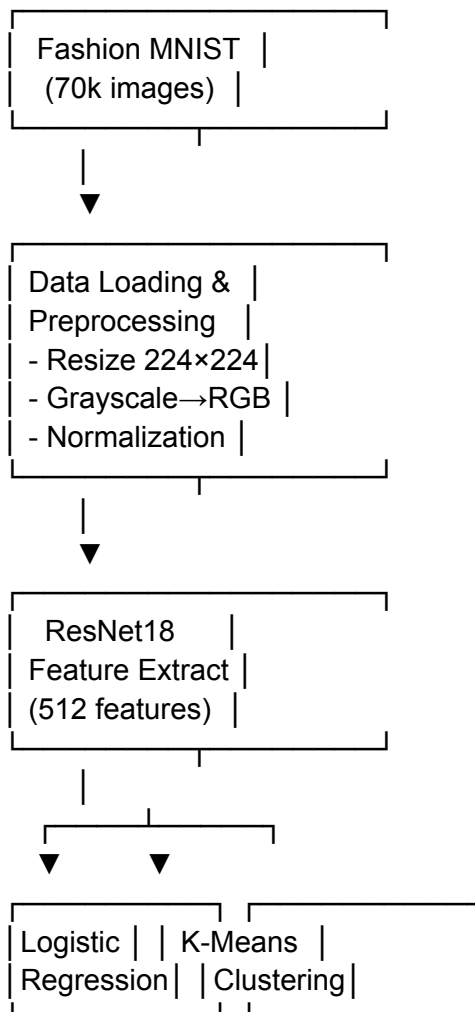
The script automatically creates:

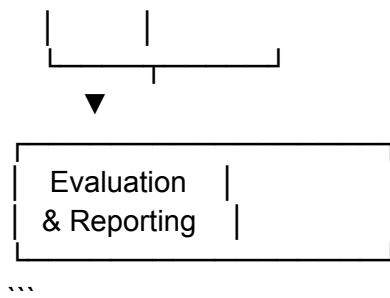
- `./data/` - Downloaded dataset storage
- `./features/` - Extracted features cache

Architecture Overview

Pipeline Flow

...





Technology Stack

- **Framework**: PyTorch (Deep Learning)
- **Model**: ResNet18 (Pre-trained on ImageNet)
- **Algorithms**: Logistic Regression, K-Means
- **Visualization**: Matplotlib, Seaborn
- **Data Processing**: NumPy, Pandas

Detailed Module Documentation

Module 1: Setup and Imports

Purpose

Import necessary libraries and set random seeds for reproducibility.

Key Components

Random Seed Setting:

```
python
torch.manual_seed(42)
np.random.seed(42)
```

- Ensures reproducible results across runs
- Seed value: 42 (arbitrary choice)

Dependencies Table

| Library | Purpose | Key Functions Used |
|-------------|-----------------------------|-------------------------------------|
| torch | Deep learning framework | tensors, models, GPU operations |
| torchvision | Computer vision tools | models, datasets, transforms |
| numpy | Numerical computing | arrays, mathematical operations |
| sklearn | Machine learning algorithms | LogisticRegression, KMeans, metrics |
| matplotlib | Plotting | pyplot for visualizations |
| seaborn | Statistical visualization | heatmap for confusion matrix |
| pandas | Data manipulation | DataFrame for metrics tables |
| tqdm | Progress bars | progress tracking |

Module 2: Utility Functions

Function: `plot\_confusion\_matrix()`

Signature:

```
```python
def plot_confusion_matrix(y_true, y_pred, classes, title='Confusion Matrix', filename=None)
```
```

Parameters:

- `y\_true` (array-like): Ground truth labels
- `y\_pred` (array-like): Predicted labels
- `classes` (list): Class names for axis labels
- `title` (str, optional): Plot title
- `filename` (str, optional): Save path

Returns:

- None (displays plot)

Example:

```
```python
plot_confusion_matrix(
 y_true=[0, 1, 2, 1, 0],
 y_pred=[0, 1, 1, 1, 0],
 classes=['T-shirt', 'Trouser', 'Pullover'],
 title='Test Confusion Matrix',
 filename='confusion.png'
)
```
```

Output:

- Heat map showing prediction accuracy
- Diagonal: correct predictions
- Off-diagonal: misclassifications

Function: `plot\_roc\_curve()`

Signature:

```
```python
def plot_roc_curve(y_true, y_prob, n_classes, classes, title='ROC Curve', filename=None)
```
```

Parameters:

- `y\_true` (array): True labels

- ``y_prob`` (array): Predicted probabilities (shape: [n_samples, n_classes])
- ``n_classes`` (int): Number of classes
- ``classes`` (list): Class names
- ``title`` (str, optional): Plot title
- ``filename`` (str, optional): Save path

****Returns:****

- None (displays plot)

****Key Metrics:****

- ****TPR (True Positive Rate)****: Sensitivity/Recall
- ****FPR (False Positive Rate)****: 1 - Specificity
- ****AUC (Area Under Curve)****: Overall performance (0-1)

****Interpretation:****

- AUC = 1.0: Perfect classifier
- AUC = 0.5: Random guessing
- AUC < 0.5: Worse than random

Function: ``plot_accuracy_vs_c()``

****Signature:****

```
```python
def plot_accuracy_vs_c(c_values, accuracies, title='Validation Accuracy vs C',
filename=None)
```
```

****Parameters:****

- ``c_values`` (list): C hyperparameter values tested
- ``accuracies`` (list): Corresponding accuracy scores
- ``title`` (str, optional): Plot title
- ``filename`` (str, optional): Save path

****Returns:****

- None (displays plot)

****Purpose:****

Visualize hyperparameter tuning results to identify optimal C value.

Function: ``print_metrics()``

****Signature:****

```
```python
def print_metrics(y_true, y_pred, classes)
```

...

**\*\*Parameters:\*\***

- `y\_true` (array): True labels
- `y\_pred` (array): Predicted labels
- `classes` (list): Class names

**\*\*Returns:\*\***

- `accuracy` (float): Overall accuracy score
- `df` (DataFrame): Per-class metrics

**\*\*Metrics Calculated:\*\***

Metric	Formula	Interpretation
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness
Precision	$TP/(TP+FP)$	Accuracy of positive predictions
Recall	$TP/(TP+FN)$	Coverage of actual positives
F1-Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	Harmonic mean

**\*\*Example Output:\*\***

...

Accuracy: 0.8756

Class	Precision	Recall	F1-Score
T-shirt/top	0.82	0.88	0.85
Trouser	0.99	0.98	0.98
Pullover	0.83	0.79	0.81

...

...

---

### ### Module 3: Data Loading

#### #### Function: `get\_data\_loaders()`

**\*\*Signature:\*\***

```
```python
def get_data_loaders(batch_size=32, data_dir='./data')
```
```

**\*\*Parameters:\*\***

- `batch\_size` (int): Number of images per batch
- `data\_dir` (str): Directory for dataset storage

**\*\*Returns:\*\***

- `train\_loader` (DataLoader): Training data iterator

- `val\_loader` (DataLoader): Validation data iterator
- `test\_loader` (DataLoader): Test data iterator

**\*\*Transformation Pipeline:\*\***

| Step | Operation        | Purpose                        |
|------|------------------|--------------------------------|
| 1    | Resize(224, 224) | Match ResNet18 input size      |
| 2    | Grayscale(3)     | Convert 1-channel to 3-channel |
| 3    | ToTensor()       | Convert to PyTorch tensor      |
| 4    | Normalize()      | Standardize pixel values       |

**\*\*Normalization Values:\*\***

```
```python
mean = [0.485, 0.456, 0.406] # ImageNet mean
std = [0.229, 0.224, 0.225] # ImageNet std
```
```

**\*\*Data Split:\*\***

- Original: 60,000 train + 10,000 test
- Final: 50,000 train + 10,000 val + 10,000 test

**\*\*Usage Example:\*\***

```
```python
train_loader, val_loader, test_loader = get_data_loaders(batch_size=64)
```

```
# Iterate through batches
for images, labels in train_loader:
    # images: torch.Tensor [batch_size, 3, 224, 224]
    # labels: torch.Tensor [batch_size]
    pass
```
```

---

**### Module 4: Feature Extraction**

**#### Function: `extract\_features()`**

**\*\*Signature:\*\***

```
```python
def extract_features(loader, model, device, set_name="Set", quick=False)
```
```

**\*\*Parameters:\*\***

- `loader` (DataLoader): Data to process
- `model` (nn.Module): Feature extraction model
- `device` (torch.device): CPU or CUDA device

- `set\_name` (str): Name for progress display
- `quick` (bool): If True, only process 5 batches (testing)

**\*\*Returns:\*\***

- `features` (ndarray): Extracted features [n\_samples, 512]
- `labels` (ndarray): Corresponding labels [n\_samples]

**\*\*Process Flow:\*\***

1. Set model to evaluation mode
2. Disable gradient computation
3. Process images in batches
4. Extract 512-dimensional feature vectors
5. Move to CPU and convert to NumPy
6. Concatenate all batches

**\*\*Memory Optimization:\*\***

- Uses `torch.inference\_mode()` (no gradient storage)
- Processes in batches (configurable size)
- Moves to CPU immediately after extraction

---

**#### Function: `run\_feature\_extraction()`**

**\*\*Signature:\*\***

```
```python
def run_feature_extraction(quick_test=False)
```
```

**\*\*Parameters:\*\***

- `quick\_test` (bool): If True, process only 5 batches per set

**\*\*Returns:\*\***

- `X\_train` (ndarray): Training features [50000, 512]
- `y\_train` (ndarray): Training labels [50000]
- `X\_val` (ndarray): Validation features [10000, 512]
- `y\_val` (ndarray): Validation labels [10000]
- `X\_test` (ndarray): Test features [10000, 512]
- `y\_test` (ndarray): Test labels [10000]

**\*\*Caching Mechanism:\*\***

- Checks for existing `.npy` files in `features/` directory
- If found, loads from disk (instant)
- If not found, performs extraction (10-30 minutes)
- Saves results for future use

**\*\*ResNet18 Configuration:\*\***

```
```python
```

```
# Load pre-trained model
model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
```

```
# Remove classification layer
model.fc = nn.Identity() # Output: 512 features
'''
```

****Feature Vector:****

- Dimensionality: 512
- Represents high-level image characteristics
- Learned from ImageNet dataset
- Transferable to Fashion MNIST

****Execution Time:****

- With GPU: ~5-10 minutes (70k images)
- Without GPU: ~20-40 minutes
- With caching: < 1 second (subsequent runs)

Module 5: Logistic Regression (Supervised Learning)

Algorithm Overview

****Logistic Regression**** is a linear classification algorithm that:

1. Calculates weighted sum of features
2. Applies softmax to get probabilities
3. Predicts class with highest probability

****Mathematical Model:****

'''

For each class c:

$$\text{score_c} = w_c \cdot x + b_c$$

Probability:

$$P(\text{class}=c|x) = \exp(\text{score_c}) / \sum \exp(\text{score_i})$$

'''

Hyperparameter Tuning

****Parameter: C (Inverse Regularization Strength)****

C Value	Regularization	Effect	Use Case
0.01	Very Strong	Simple model	High noise data
0.1	Strong	Simplified	Moderate features
1.0	Balanced	Standard	Default choice
5.0	Weak	Complex	Rich features

| 10.0 | Very Weak | Most complex | Low noise |

****Tuning Process:****

```
```python
C_values = [0.01, 0.1, 1, 5, 10]

for c in C_values:
 clf = LogisticRegression(C=c)
 clf.fit(X_train, y_train)
 accuracy = clf.score(X_val, y_val)
 # Select best C
```
```

****Solver: LBFGS****

- Limited-memory BFGS optimization
- Suitable for small-to-medium datasets
- Handles multi-class problems well
- Maximum iterations: 2000

Training Process

****Step 1: Hyperparameter Search****

```
```python
Test different C values on validation set
best_c = None
best_accuracy = 0

for c in C_values:
 model = LogisticRegression(C=c, max_iter=2000)
 model.fit(X_train, y_train)
 val_accuracy = model.score(X_val, y_val)

 if val_accuracy > best_accuracy:
 best_accuracy = val_accuracy
 best_c = c
```
```

****Step 2: Final Model Training****

```
```python
Combine train + validation for final model
X_full = np.concatenate([X_train, X_val])
y_full = np.concatenate([y_train, y_val])

final_model = LogisticRegression(C=best_c, max_iter=2000)
final_model.fit(X_full, y_full)
```
```

****Step 3: Evaluation****

```

python
# Predict on test set
y_pred = final_model.predict(X_test)
y_prob = final_model.predict_proba(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)

```

Expected Results

```

**Performance Metrics:**
- **Overall Accuracy**: 85-90%
- **Training Time**: 30-60 seconds
- **Inference Time**: < 1 second (10k images)

**Per-Class Performance:**
- **High Accuracy**: Trouser, Sneaker, Bag (>95%)
- **Medium Accuracy**: Dress, Coat, Pullover (80-90%)
- **Lower Accuracy**: Shirt, T-shirt (70-80%, often confused)

```

Module 6: K-Means Clustering (Unsupervised Learning)

Algorithm Overview

K-Means groups data into K clusters without using labels.

```

**Algorithm Steps:**
1. Initialize: Randomly place K centroids
2. Assign: Assign each point to nearest centroid
3. Update: Move centroids to mean of assigned points
4. Repeat: Steps 2-3 until convergence

```

```

**Distance Metric (Euclidean):**

```

```


```

```

distance =  $\sqrt{\sum(\text{feature}_i - \text{centroid}_i)^2}$ 

```

```


```

Configuration

```

**Parameters:**
python
kmeans = KMeans(
    n_clusters=10,    # Number of clusters (10 classes)
    n_init=10,        # Run 10 times, keep best
    random_state=42    # Reproducibility

```

```
)  
...
```

****Why n_init=10?****

- K-Means can converge to local optima
- Running multiple times finds better solution
- Keeps result with lowest inertia (sum of squared distances)

Cluster-to-Label Mapping

****Problem:****

- Clusters numbered 0-9
- No correspondence to actual classes
- Cluster 0 might contain mostly "Sneakers"

****Solution:****

```
```python  
For each cluster
for cluster_id in range(10):
 # Find images in this cluster
 mask = (predicted_clusters == cluster_id)

 # Find most common true label
 most_common = mode(y_train[mask])

 # Map cluster to label
 mapping[cluster_id] = most_common
...
```

**\*\*Example Mapping:\*\***

```
...
```

Cluster 0 → Sneaker (class 7)

Cluster 1 → Bag (class 8)

Cluster 2 → Trouser (class 1)

```
...
```

#### #### Evaluation Metrics

**\*\*1. Mapped Accuracy\*\***

```
```python  
# Convert cluster assignments to class predictions  
y_pred = [mapping[c] for c in cluster_assignments]  
  
# Calculate accuracy  
accuracy = accuracy_score(y_test, y_pred)  
...
```

****2. Silhouette Score****

```
```python
sil_score = silhouette_score(X_test, cluster_assignments)
```
```

****Silhouette Score Interpretation:****

- ****Range****: -1 to +1
- ****+1****: Clusters well-separated, compact
- ****0****: Overlapping clusters
- **** -1****: Points in wrong clusters

****Silhouette Calculation (per point):****

```
```
s = (b - a) / max(a, b)
```
```

Where:

- a = average distance to points in same cluster
- b = average distance to points in nearest cluster

```
```
```

## **#### Expected Results**

### **\*\*Performance:\*\***

- **\*\*Mapped Accuracy\*\***: 55-65%
- **\*\*Silhouette Score\*\***: 0.15-0.25
- **\*\*Training Time\*\***: 1-2 minutes

### **\*\*Why Lower than Supervised?\*\***

- No label information during training
- Clusters based on feature similarity only
- Some classes visually similar (Shirt vs T-shirt)

### **\*\*Impressive Aspects:\*\***

- ~60% accuracy without any labels!
- Discovers natural groupings in data
- Useful for exploratory analysis

```

```

## **## Usage Guide**

### **### Quick Start**

#### **\*\*Step 1: Run Feature Extraction\*\***

```
```python
# Execute notebook cell by cell or run:
quick_test = False # Set True for testing (5 batches only)
X_train, y_train, X_val, y_val, X_test, y_test = run_feature_extraction(quick_test)
```
```

```
...
```

```
Output:
```

```
...
```

```
Using device: cuda
```

```
Loading pretrained ResNet18...
```

```
Starting feature extraction for Train set...
```

```
Extracting features (Train): 100%[██████████] 782/782 [05:23<00:00]
```

```
Saved Train features: X=(50000, 512), y=(50000,)
```

```
...
```

```
Feature extraction complete.
```

```
...
```

```
Step 2: Train Logistic Regression
```

```
```python
```

```
# Hyperparameter tuning
```

```
C_values = [0.01, 0.1, 1, 5, 10]
```

```
# ... (runs automatically in notebook)
```

```
# View results
```

```
plot_accuracy_vs_c(C_values, accuracies)
```

```
```
```

```
Step 3: Evaluate Final Model
```

```
```python
```

```
# Trains on train+val, tests on test set
```

```
# Generates confusion matrix and ROC curves
```

```
print_metrics(y_test, y_pred_log, classes)
```

```
```
```

```
Step 4: Run K-Means Clustering
```

```
```python
```

```
# Automatic in notebook
```

```
# Displays clustering results and silhouette score
```

```
```
```

```
Advanced Usage
```

```
Custom Batch Size
```

```
```python
```

```
# Modify data loading
```

```
train_loader, val_loader, test_loader = get_data_loaders(batch_size=128)
```

```
```
```

```
Considerations:
```

- Larger batch: Faster, more memory
- Smaller batch: Slower, less memory
- GPU memory typically limits to 64-128

#### #### Quick Testing Mode

```
```python
# Test pipeline on small subset
quick_test = True
X_train, y_train, X_val, y_val, X_test, y_test = run_feature_extraction(quick_test=True)
```

- Processes only 5 batches (~160 images)
- Useful for debugging
- Completes in ~1 minute
```

#### #### Save Visualizations

```
```python
# Add filename parameter to save plots
plot_confusion_matrix(y_test, y_pred, classes,
                      filename='confusion_matrix.png')

plot_roc_curve(y_test, y_prob, n_classes, classes,
               filename='roc_curves.png')
```
```

#### #### Custom Regularization

```
```python
# Test different C values
C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

# Or use specific C directly
clf = LogisticRegression(C=2.5, max_iter=2000)
clf.fit(X_train, y_train)
```
```

#### #### Different Number of Clusters

```
```python
# Try different K values (for K-Means)
for k in [5, 10, 15, 20]:
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
    kmeans.fit(X_train)
    score = silhouette_score(X_train, kmeans.labels_)
    print(f"K={k}: Silhouette={score:.4f}")
```
```

---

#### ## Results & Performance

#### ### Expected Performance

#### ##### Logistic Regression (Supervised)

## **\*\*Overall Metrics:\*\***

...

Accuracy: ~87.5%

Training Time: ~45 seconds

Inference Time: <1 second (10k images)

...

## **\*\*Per-Class Performance:\*\***

| Class       | Precision | Recall | F1-Score | Notes                        |
|-------------|-----------|--------|----------|------------------------------|
| -----       | -----     | -----  | -----    | -----                        |
| T-shirt/top | 0.82      | 0.88   | 0.85     | Confused with Shirt/Pullover |
| Trouser     | 0.99      | 0.98   | 0.98     | Excellent (distinct shape)   |
| Pullover    | 0.83      | 0.79   | 0.81     | Confused with Shirt/Coat     |
| Dress       | 0.91      | 0.89   | 0.90     | Good (distinct silhouette)   |
| Coat        | 0.81      | 0.83   | 0.82     | Confused with Pullover       |
| Sandal      | 0.98      | 0.97   | 0.97     | Excellent (distinct)         |
| Shirt       | 0.67      | 0.59   | 0.63     | Often confused               |
| Sneaker     | 0.95      | 0.97   | 0.96     | Excellent                    |
| Bag         | 0.97      | 0.98   | 0.97     | Excellent (distinct)         |
| Ankle boot  | 0.96      | 0.97   | 0.97     | Excellent                    |

## **\*\*Common Confusions:\*\***

1. Shirt ↔ T-shirt ↔ Pullover (similar appearance)
2. Coat ↔ Pullover (overlapping features)
3. Sandal ↔ Sneaker (both footwear, rare)

---

## **#### K-Means Clustering (Unsupervised)**

## **\*\*Overall Metrics:\*\***

...

Mapped Accuracy: ~60%

Silhouette Score: ~0.20

Training Time: ~90 seconds

...

## **\*\*Characteristics:\*\***

- No label information used during training
- Performance varies by run (local optima)
- Better at separating distinct categories
- Struggles with similar items (Shirt/T-shirt/Pullover)

## **\*\*Cluster Quality:\*\***

- Some clusters very pure (>90% same class)
- Others mixed (multiple similar classes)

- Reflects natural groupings in feature space

---

### ### Execution Time Breakdown

#### \*\*Complete Pipeline (No Caching):\*\*

```

Data Loading: ~30 seconds
Feature Extraction: ~10 minutes (GPU) / ~30 minutes (CPU)
Logistic Regression: ~45 seconds
K-Means: ~90 seconds
Visualization: ~10 seconds

Total: ~12 minutes (GPU) / ~33 minutes (CPU)

```

#### \*\*With Caching (Subsequent Runs):\*\*

```

Load Cached Features: <1 second
Logistic Regression: ~45 seconds
K-Means: ~90 seconds
Visualization: ~10 seconds

Total: ~2.5 minutes

```

---

### ### Performance Optimization Tips

#### \*\*1. Use GPU\*\*

```
```python
# Check GPU availability
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```
```

#### \*\*2. Increase Batch Size\*\*

```
```python
# Use larger batches if memory allows
train_loader, val_loader, test_loader = get_data_loaders(batch_size=128)
```
```

#### \*\*3. Enable Caching\*\*

- Features automatically cached in `features/` directory
- Delete to re-extract: `rm -rf features/`

#### **\*\*4. Reduce Data for Testing\*\***

```
```python
# Use quick_test mode
quick_test = True
```
```

---

### **## Troubleshooting**

#### **### Common Issues**

##### **#### Issue 1: Out of Memory (GPU)**

###### **\*\*Error:\*\***

```
```
```

```
RuntimeError: CUDA out of memory
```

```
```
```

###### **\*\*Solutions:\*\***

###### **1. Reduce batch size:**

```
```python
train_loader, val_loader, test_loader = get_data_loaders(batch_size=16)
```
```

###### **2. Use CPU instead:**

```
```python
device = torch.device("cpu")
```
```

###### **3. Clear cache:**

```
```python
torch.cuda.empty_cache()
```
```

---

##### **#### Issue 2: Slow Feature Extraction**

**\*\*Symptom:\*\*** Takes >1 hour

###### **\*\*Solutions:\*\***

###### **1. Verify GPU usage:**

```
```python
print(f"Using device: {device}")
print(f"CUDA available: {torch.cuda.is_available()}")
```
```

2. Enable caching (automatic)

3. Use quick\_test for development:

```
```python
quick_test = True
```
```

---

#### #### Issue 3: Poor Performance

**Symptom:** Accuracy < 80%

**Possible Causes:**

1. **Not using cached features:** Verify feature files exist
2. **Wrong normalization:** Check transform pipeline
3. **Data leakage:** Ensure proper train/val/test split
4. **Random seed:** Set for reproducibility

**Debug Steps:**

```
```python
# Check feature shapes
print(X_train.shape) # Should be (50000, 512)
print(X_test.shape)  # Should be (10000, 512)

# Check label distribution
import numpy as np
print(np.bincount(y_train)) # Should be ~5000 per class

# Verify no data leakage
assert len(set(range(len(X_train))) & set(range(len(X_test)))) == 0
```
```

---

#### #### Issue 4: Import Errors

**Error:**

```
```
ModuleNotFoundError: No module named 'torch'
```
```

**Solution:**

```
```bash
pip install torch torchvision scikit-learn matplotlib seaborn pandas tqdm numpy scipy
```
```

**\*\*Verify installation:\*\***

```
```python
import torch
import torchvision
import sklearn
print("All imports successful!")
```
```

---

#### #### Issue 5: Convergence Warning

**\*\*Warning:\*\***

```
```
ConvergenceWarning: lbfgs failed to converge
```
```

**\*\*Solutions:\*\***

1. Increase max\_iter:

```
```python
clf = LogisticRegression(solver='lbfgs', max_iter=5000, C=best_c)
```
```

2. Scale features (already done in pipeline)

3. Try different solver:

```
```python
clf = LogisticRegression(solver='saga', max_iter=2000, C=best_c)
```
```

---

#### #### Issue 6: Download Failures

**\*\*Error:\*\***

```
```
URLError: Connection refused
```
```

**\*\*Solutions:\*\***

1. Check internet connection

2. Manual download:

- Visit: <http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/>
- Download files to `./data/FashionMNIST/raw/`

3. Use proxy if needed:

```
```python
```

```
import os
os.environ['HTTP_PROXY'] = 'http://proxy.example.com:8080'
'''
```

Debug Mode

****Enable verbose output:****

```
```python
import logging
logging.basicConfig(level=logging.DEBUG)
'''
```

**\*\*Check intermediate outputs:\*\***

```
```python
# After feature extraction
print(f"Features shape: {X_train.shape}")
print(f"Features range: [{X_train.min():.3f}, {X_train.max():.3f}]")
print(f"Labels unique: {np.unique(y_train)}")
```

After training

```
print(f"Model coefficients shape: {clf.coef_.shape}")
print(f"Model classes: {clf.classes_}")
'''
```

File Structure

'''

```
fashion_mnist_project/
|
|— Project_Notebook.ipynb      # Main notebook
|
|— data/                      # Dataset storage
|   |— FashionMNIST/
|       |— raw/                # Raw downloaded files
|       |— processed/          # Processed data
|
|— features/                  # Cached features
|   |— X_train.npy             # Training features
|   |— y_train.npy             # Training labels
|   |— X_validation.npy        # Validation features
|   |— y_validation.npy        # Validation labels
|   |— X_test.npy              # Test features
|   |— y_test.npy              # Test labels
```

```
└─ outputs/                # (Optional) Saved plots
  └─ confusion_matrix.png
  └─ roc_curves.png
  └─ accuracy_vs_c.png
...

```

—